



The Bitless (Gruppo 18):

Lorenzo Battistella (2103116), Edoardo De Piccoli (2101055), Davide Facco (2087852),
Anna Marini (2110986), Andrea Menegaldo (2116426), Samuele Vendramin (2111934),
Simone Zecchinato (2113189)

Informazioni documento			
Versione	Data	Stato	Destinatari
0.5.0	2026-04-29	In stesura	The Bitless, prof. Tullio Vardanega, prof. Riccardo Cardin

Contatti: thebitless.swe@gmail.com

Versione	Data	Autore	Verificatore	Approvatore	Descrizione
0.5.0	2026-04-29	A. Marini	S. Zecchinato	-	Processi organizzativi
0.4.0	2026-04-20	S. Zecchinato	E. De Piccoli	-	Verifica
0.3.0	2026-04-18	S. Zecchinato	E. De Piccoli	-	Gestione delle configurazioni e inizio verifica
0.2.0	2026-04-09	S. Zecchinato	S. Vendramin	-	Inizio Processi di supporto
0.1.0	2026-04-03	A. Marini	S. Vendramin	-	Introduzione e Processi primari

Indice

1 Introduzione

1.1 Scopo del documento

Questo documento delinea le modalità operative, denominate *Way of Working_G*, adottate dal team **The Bitless**. Il suo scopo è stabilire un protocollo comune di lavoro: una serie di norme e best practice che ogni componente del gruppo deve rispettare. Questo garantisce che lo sviluppo proceda in modo fluido, ordinato e con uno standard qualitativo elevato.

Il documento recepisce le direttive del *Regolamento del Progetto Didattico* e si ispira allo standard internazionale **ISO/IEC 12207:1995**. Tale norma individua tre principali tipologie di processi:

- **Processi Primari:** tutto ciò che serve a creare il prodotto, dalla raccolta dei requisiti fino alla manutenzione finale;
- **Processi di Supporto:** attività trasversali che garantiscono la solidità del lavoro, come la gestione della configurazione e le attività di test;
- **Processi Organizzativi:** compiti dedicati alla gestione delle risorse, alla formazione interna e al costante miglioramento dell'infrastruttura di squadra.

In linea con lo standard, è stato applicato un processo di **personalizzazione (tailoring_G)**, selezionando esclusivamente gli elementi necessari al nostro contesto specifico, evitando sovraccarichi burocratici. Il documento seguirà un approccio **incrementale**, venendo aggiornato e perfezionato man mano che il team acquisisce nuove competenze e feedback.

1.2 Finalità del prodotto

In sinergia con la proponente **Zucchetti S.p.A.**, il gruppo sta realizzando **Second Brain**, una piattaforma digitale per la gestione della conoscenza basata sul formato **Markdown_G**. L'applicativo offre una doppia visualizzazione: un'area di scrittura pulita e un'anteprima immediata del risultato finale.

Il valore aggiunto del progetto risiede nell'integrazione di **LLM_G** per abilitare il concetto di **"Distant Writing"_G**. In questo modello, l'utente assume il ruolo di progettista del testo e coordinatore del "plot", mentre l'intelligenza artificiale interviene iterativamente nella stesura dei contenuti, supportando operazioni di riscrittura, riassunto, traduzione ed estrazione di concetti rilevanti.

Il sistema non si limita a scrivere, ma analizza: sfruttando la tecnica dei **"Sei Cappelli per Pensare"**, l'assistente virtuale è capace di esaminare i testi da diverse prospettive, offrendo un supporto decisionale strutturato.

Il termine ultimo per la consegna è fissato per il **30 agosto 2026**, con un budget operativo previsto di **12.860,00 €**.

1.3 Glossario e chiarezza terminologica

La progettazione di un software è un'attività complessa che richiede un linguaggio comune per evitare fraintendimenti. Per questo motivo, abbiamo redatto un **Glossario** dedicato. Ogni termine tecnico o specifico del dominio che richiede un approfondimento è contrassegnato da una "G" a pedice, segnalando che la sua definizione completa è consultabile nel documento esterno. Ad esempio: *parola_G*.

1.4 Riferimenti

1.4.1 Riferimenti normativi

- **Capitolato C6 - Second Brain:**
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C6.pdf>
- **Regolamento del Progetto Didattico:**
<https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/PD1.pdf>
- **Standard ISO/IEC 12207:1995:**
www.math.unipd.it/~tullio/IS-1/2009/Approfondimenti/ISO_12207-1995.pdf

1.4.2 Riferimenti informativi

- **Glossario di Progetto:**
https://thebitless.github.io/RTB/doc_interna/glossario/glossario.pdf

2 Processi Primari

In conformità allo standard **ISO/IEC 12207:1995**, i processi primari costituiscono le attività essenziali che coprono l'intero ciclo di vita del software. Essi definiscono le responsabilità operative e gestionali delle figure coinvolte nella realizzazione del progetto.

Il gruppo **The Bitless** ha selezionato i seguenti processi come cardini del proprio *Way of Working_G*:

- **Fornitura** (Supply Process);
- **Sviluppo** (Development Process).

2.1 Fornitura

Il processo di fornitura regola il rapporto tra il gruppo (fornitore) e il committente (docenti e azienda). In questa fase, la priorità è la gestione delle aspettative e il rispetto dei vincoli economici e temporali.

2.1.1 Gestione della comunicazione

Per garantire un coordinamento efficace delle attività di fornitura, sono stati selezionati i seguenti strumenti:

- **GitHub_G**: utilizzato per la gestione del *Backlog_G* e del sistema di *ticketing_G*, fondamentali per monitorare l'avanzamento dei lavori e la pianificazione tramite Diagrammi di Gantt_G;
- **Discord e WhatsApp**: canali dedicati rispettivamente alle riunioni sincrone e alle comunicazioni rapide asincrone tra i membri;
- **Google Mail**: canale ufficiale per la corrispondenza formale. Viene impiegato esclusivamente per le comunicazioni scritte verso i referenti esterni e le istituzioni, garantendo tracciabilità;
- **Google Meet**: piattaforma scelta per le conferenze da remoto con l'azienda proponente. Questo strumento permette di svolgere revisioni periodiche e presentazioni dello stato di avanzamento in un ambiente stabile e condiviso.

2.1.2 Attività previste

In linea con lo standard, il processo di fornitura si articola in sette attività principali:

1. **Initiation:** rappresenta il punto di partenza in cui il gruppo compie un'analisi approfondita delle richieste avanzate dal proponente (**RFP_G**). Gli studenti valutano i requisiti per determinare la fattibilità_G del progetto e verificare che il team possieda le competenze tecniche necessarie per procedere.
2. **Preparation of response:** fase dedicata alla redazione della proposta progettuale. In questo stadio viene definito il **Tailoring_G** dello standard, selezionando esclusivamente i processi e i compiti realmente funzionali agli obiettivi del gruppo, mantenendo il rigore metodologico richiesto.
3. **Contract:** accordo formale tra il gruppo, il docente e il tutor aziendale. Vengono definiti con precisione i **deliverable_G**, le scadenze temporali e i criteri di accettazione per pervenire alla formalizzazione del contratto sui requisiti e alla stipula definitiva.
4. **Planning:** fase in cui il team organizza il lavoro internamente definendo il **framework_G** di gestione. Si sceglie il modello di sviluppo, si distribuiscono i compiti, si allocano le risorse e si effettua un'analisi preventiva dei rischi tecnici e temporali.
5. **Execution and control:** attuazione operativa dei piani definiti e produzione vera e propria di codice e documentazione. Il gruppo monitora costantemente lo stato di avanzamento del lavoro, il rispetto del budget_G e delle tempistiche, gestendo le criticità in itinere per garantire la stabilità del progetto.
6. **Review and evaluation:** consiste negli incontri periodici con il tutor aziendale per mostrare avanzamenti e ricevere feedback. Vengono applicati processi di **Verifica_G** e **Validazione_G** per dimostrare che il software soddisfi i requisiti e sia pronto per l'accettazione finale.
7. **Delivery and completion:** fase conclusiva di consegna del prodotto e della documentazione. Include il supporto durante i test di accettazione e l'eventuale formazione del personale aziendale per l'installazione e l'utilizzo del software, secondo i termini contrattuali stabiliti.

2.1.3 Documentazione fornita

Il gruppo The Bitless si impegna nella redazione e nel costante aggiornamento della documentazione di progetto. Tali documenti sono essenziali per garantire la continuità operativa del team e una comunicazione trasparente con i soggetti esterni.

2.1.3.1 Documenti ad Uso Esterno Questi artefatti sono destinati al committente_G e ai proponenti per formalizzare accordi, piani e risultati tecnici:

- **Lettera di Candidatura:** istanza formale con cui il gruppo manifesta l'interesse per il capitolato C6 proposto da **Zucchetti S.p.A.**. Illustra le ragioni della scelta e fornisce le coordinate per accedere ai *repository_G* ufficiali del progetto.
- **Analisi dei Capitolati:** documento di analisi comparativa che espone le motivazioni tecniche e logistiche che hanno portato alla selezione del progetto **Second Brain** rispetto alle altre proposte disponibili. Contiene l'analisi in dettaglio di ogni capitolato, descrivendone l'obiettivo, le tecnologie adottate e le considerazioni emerse durante il confronto nel gruppo.

- **Preventivo costi e assunzione impegni:** documento che ufficializza l'impegno operativo dei membri del team, stabilito in un monte ore produttivo di 93 ore per ciascun componente. Include il piano economico e la strategia di rotazione dei ruoli_G ogni due settimane, volta a garantire un'esperienza formativa completa e un carico di lavoro bilanciato tra i componenti.
- **Analisi dei Requisiti:** descrive in modo dettagliato le funzionalità e i vincoli del sistema **Second Brain** (Capitolato C6). L'identificazione delle necessità avviene tramite lo studio dei **Casi d'Uso**_G, corredati da diagrammi illustrativi e matrici di tracciamento_G per assicurare la totale copertura dei requisiti concordati.
- **Piano di Progetto:** elaborato di pianificazione strategica gestito con metodologia incrementale_G. Analizza la ripartizione temporale delle attività, la gestione delle risorse e dei rischi, fornendo un monitoraggio costante dell'avanzamento e del bilancio dei costi_G sostenuti.
- **Piano di Qualifica:** specifica i protocolli di **Verifica** e **Validazione** adottati. Contiene il dettaglio dei test eseguiti sul prodotto, le metriche di qualità applicate e i resoconti dei risultati ottenuti per garantire la conformità agli standard.
- **Verbale Esterno:** resoconto formale delle riunioni tenutesi con soggetti esterni al team, atto a cristallizzare decisioni, feedback e scadenze concordate con il committente_G.

2.1.3.2 Documenti ad Uso Interno Si tratta di strumenti operativi riservati ai membri del gruppo, fondamentali per mantenere la coerenza metodologica:

- **Norme di Progetto:** documento cardine che definisce il *Way of Working*_G del gruppo sulla base dello standard **ISO/IEC 12207:1995**. Al suo interno sono fissate le procedure per ogni attività, le convenzioni per scrivere il codice, gli strumenti da utilizzare e le tecniche per migliorare costantemente la qualità del lavoro prodotto.
- **Glossario:** dizionario condiviso creato per raccogliere e spiegare i termini tecnici legati allo specifico dominio_G del progetto. Il suo scopo è evitare malintesi, assicurando che ogni parola abbia un unico significato per tutti i membri; i termini presenti in questa lista sono contrassegnati nei documenti dal pedice "G".
- **Verbale Interno:** contiene il riassunto delle discussioni e l'elenco delle decisioni prese durante le riunioni tra i componenti di **The Bitless**, permettendo di tenere traccia dell'evoluzione del progetto e delle responsabilità assegnate a ciascuno.

2.2 Sviluppo

Il processo di sviluppo definisce le attività tecniche necessarie per la progettazione, codifica, installazione e accettazione del software.

2.2.1 Attività previste

In conformità alla subclause 5.3 dello standard, il team esegue le seguenti 13 attività:

1. **Process implementation:** in questa fase vengono stabiliti i piani operativi, gli strumenti di sviluppo e i linguaggi di programmazione da adottare.

2. **System requirements analysis:** analisi degli usi previsti per definire i **requisiti funzionali**_G, di capacità e di sicurezza, traducendo le richieste in specifiche tecniche documentate e realizzabili.
3. **System architectural design:** definizione dell'architettura di alto livello, individuando le componenti software, hardware e le interfacce esterne. In questo stadio si stabilisce come i diversi moduli interagiranno tra loro per soddisfare l'architettura complessiva.
4. **Software requirements analysis:** si definiscono le specifiche di ogni componente, comprese le interfacce utente e i **requisiti di qualificazione**_G necessari per validare il corretto funzionamento di ogni parte del sistema.
5. **Software architectural design:** trasformazione dei requisiti in una struttura software gerarchica. Si definisce il design preliminare delle interfacce e dello schema del **database**_G.
6. **Software detailed design:** raffinamento dei componenti in **unità software**_G elementari. Viene completato il design di dettaglio per permettere la successiva fase di codifica individuale.
7. **Software coding and testing:** realizzazione effettiva del codice e delle basi di dati. Lo sviluppatore esegue i **test unitari**_G per verificare che ogni singola unità rispetti le specifiche.
8. **Software integration:** si procede all'assemblaggio dei moduli sviluppati individualmente dai membri del team. I componenti vengono aggregati e verificati collettivamente seguendo un piano di Integrazione_G, con l'obiettivo di accertare che il flusso di dati e le interfacce tra le varie parti del sistema operino in modo armonico e senza conflitti.
9. **Software qualification testing:** esecuzione di test_G formali per dimostrare la conformità del software integrato ai requisiti stabiliti.
10. **System integration:** il software viene unito all'hardware e agli altri sistemi esterni. Il sistema complessivo viene testato durante l'assemblaggio per garantirne la coerenza globale.
11. **System qualification testing:** collaudo finale del sistema integrato per verificare la piena conformità ai requisiti originali e la prontezza per la consegna definitiva.
12. **Software installation:** installazione del prodotto nell'ambiente di destinazione. Si verifica che il codice e il database siano inizializzati correttamente e operino come previsto.
13. **Software acceptance support:** assistenza al committente durante i test_G di accettazione formale, fornendo il supporto tecnico necessario per l'utilizzo del sistema.

2.2.2 Analisi dei Requisiti

L'attività di analisi è fondamentale per supportare il lavoro di progettisti e programmatori, fornendo le basi per l'architettura e la codifica.

La redazione di questo documento è affidata agli Analisti_G, i quali hanno il compito di individuare, classificare e garantire la tracciabilità_G dei requisiti che l'applicativo deve soddisfare. Il processo segue un approccio analitico che deriva i requisiti direttamente dai Casi d'Uso_G: scenari d'interazione tra un utente e il sistema, finalizzati al raggiungimento di uno scopo specifico. L'individuazione di tali scenari avviene attraverso sessioni di Brainstorming_G interno al gruppo e viene perfezionata grazie ai Feedback_G costanti ricevuti dall'azienda proponente.

Per garantire chiarezza e completezza, il documento è organizzato nelle seguenti sezioni:

1. **Introduzione:** definisce lo scopo del prodotto, il glossario specifico e i riferimenti (normativi e informativi) seguiti durante l'analisi.
2. **Descrizione generale:** offre una panoramica del sistema, descrivendo il contesto d'uso, le caratteristiche degli utenti previsti e gli eventuali vincoli tecnologici o logistici.
3. **Casi d'Uso_G:** l'elenco gerarchico degli scenari d'uso, corredati da diagrammi UML_G e descrizioni testuali che specificano pre-condizioni, post-condizioni e flussi degli eventi (principali e alternativi).
4. **Requisiti_G:** classificazione dettagliata dei requisiti emersi, suddivisi per tipologia:
 - **Obbligatorî, Desiderabili e Opzionali** (per priorità);
 - **Funzionali, Qualitativi e di Vincolo** (per natura tecnica).
5. **Tabelle di Tracciamento_G:** matrici che collegano univocamente i requisiti alle loro fonti e viceversa, assicurando che ogni richiesta del committente sia stata presa in carico.

2.2.2.1 Casi d'Uso Viene utilizzata la nomenclatura: **UC[Principale].[Secondario]**. Ogni caso è accompagnato da un identificativo univoco, un titolo esplicativo e una descrizione del flusso di eventi.

2.2.2.2 Requisiti Ogni requisito è tracciato con il formato: **R[Tipologia]-[Numero]**. Le tipologie si dividono in **F** (Funzionale), **Q** (Qualità) e **V** (Vincolo).

3 Processi di supporto

I processi di supporto comprendono le attività trasversali necessarie a garantire il corretto svolgimento dei processi primari e organizzativi, assicurando che ogni artefatto prodotto sia conforme agli standard di qualità prefissati.

3.1 Documentazione

La documentazione risulta fondamentale per supportare qualsiasi processo primario per il quale è necessario avere nota delle decisioni prese dal gruppo circa gli aspetti del progetto.

Il processo di documentazione definisce le regole per la redazione, la revisione e la manutenzione di tutti i documenti prodotti dal gruppo *The Bitless*.

3.1.1 Strumenti scelti

Per la redazione dei documenti lo strumento scelto è L^AT_EX_G. Si tratta di un linguaggio di markup molto utilizzato in ambito accademico per redigere documenti formali in maniera efficiente. Esso permette infatti di integrare librerie, templates e di creare comandi che semplificano di molto la scrittura dei documenti. Tutti i sorgenti dei documenti sono reperibili nella repository GitHub_G affiancati dalla compilazione in PDF.

3.1.2 Versionamento

Per il versionamento della documentazione lo strumento scelto è stato Git_G a cui applicheremo le seguenti norme per un corretto ed uniforme versionamento.

3.1.2.1 Branch

Ogni membro del gruppo che lavora su un documento, o su una parte di esso, lavora su un branch_G dedicato. Quando il documento è inteso dall'autore come pronto per essere unito al main branch_G l'autore apre una pull request_G indirizzata ai verificatori che provvederanno a verificare le modifiche. Se la verifica ha esito positivo la pull request viene approvata e il branch viene chiuso, altrimenti il documento rimane nel branch per le ulteriori modifiche.

3.1.2.2 Stato

Ogni documento riporta nel frontespizio lo stato in cui lo stesso si trova:

- **Stesura:** il documento è in fase di stesura, non è ritenuto valido come *ground truth*_G.
- **Revisione:** il documento è in fase di revisione, non è consentita la modifica a meno che il documento non ritorni in **Stesura**. Non è considerabile come *ground truth*.
- **Approvato:** il documento è stato approvato, non è consentita la modifica per nessun motivo e viene ritenuto *ground truth* per il progetto.

3.1.2.3 Nomenclatura

Ogni documento deve riportare un nome significativo che permetta a chiunque consulti la repository di capire immediatamente di cosa parla il documento senza doverlo aprire.

- **Verbali Interni:** VI.AAAA-MM-GG_descrizione-breve.pdf
- **Verbali Esterni:** VE.AAAA-MM-GG_descrizione-breve.pdf o, se redatto prima della candidatura, VE.AAAA-MM-GG_azienda.pdf
- **Documenti Generali:** nome-documento.vX.Y.Z.pdf

3.1.2.4 Commit e Pull Request

Ogni Commit_G e ogni Pull Request_G devono apportare modifiche significative e devono quindi essere fatti in maniera accorta. Le descrizioni ai commit e alle pull request devono avere carattere significativo, s'intende quindi che il commento deve poter essere letto e compreso da qualunque membro del team senza dover andare ad osservare il contenuto dell'azione.

3.1.3 Struttura generale dei documenti

- **Prima pagina (Frontespizio):** Costituisce la copertina del documento e deve riportare, in ordine di posizionamento:
 - Logo ufficiale del gruppo **The Bitless**;
 - Titolo identificativo del documento;
 - Nome del gruppo e relativo numero identificativo;
 - Elenco completo dei componenti del team, comprensivo di Nome, Cognome e numero di matricola UniPd;
 - Informazioni di controllo: versione corrente, data di rilascio della versione, stato del documento e destinatari;
 - Recapito e-mail ufficiale del gruppo.

- **Registro delle modifiche:** Collocato a partire dalla seconda pagina, consiste in una tabella organizzata secondo una logica **LIFO**_G, in cui la versione più recente appare in cima. Per ogni revisione devono essere indicati: versione, data, autore, verificatore_G e una descrizione sintetica delle variazioni apportate rispetto alla release_G precedente.
- **Indice:** Presenta l'organizzazione gerarchica_G del documento, elencando tutte le sezioni e sottosezioni. Ogni voce deve essere configurata come collegamento ipertestuale_G per permettere la navigazione diretta verso la parte del documento associata.
- **Intestazione:** Ciascuna pagina deve presentare un'intestazione costante che riporti il nome del gruppo **The Bitless** e il titolo del documento in oggetto.

3.1.3.1 Struttura dei verbali

I verbali, sia interni che esterni, oltre a rispettare i criteri generali descritti nella sezione ??, devono presentare una struttura fissa composta dalle seguenti sezioni:

- **Informazioni sulla riunione:** Paragrafo descrittivo contenente i dati logistici dell'incontro:
 - Data di svolgimento;
 - Ora di inizio e ora di conclusione;
 - Modalità di svolgimento.
- **Presenze:** Tabella riassuntiva riportante lo stato dei componenti del gruppo:
 - Nome e Cognome di tutti i membri del team;
 - Stato di presenza alla riunione (Presente/Assente).
- **Ordine del giorno:** Elenco puntato dei punti di discussione previsti per la sessione.
- **Discussione e sintesi:** Sezione discorsiva dedicata all'approfondimento degli argomenti trattati e al resoconto del brainstorming_G effettuato.
- **Attività da svolgere:** Tabella dedicata alla pianificazione e assegnazione_G dei nuovi task emersi, con relativi riferimenti alle Issue_G di GitHub_G.

3.1.4 Ciclo di vita dei documenti

3.1.4.1 Redazione dei documenti

La stesura e l'aggiornamento sistematico dei documenti è affidata al membro del gruppo che ricopre, nel periodo di riferimento, il ruolo di *Amministratore*_G. Le informazioni necessarie alla redazione vengono estrapolate dai verbali delle riunioni e dalle comunicazioni intercorse tra i componenti del team.

3.1.4.2 Redazione dei verbali

La responsabilità della stesura dei verbali, sia interni (VI) che esterni (VE), spetta a chi ricopre il ruolo di *Responsabile*_G. I contenuti vengono sintetizzati a partire dalle discussioni avvenute durante i meeting interni o dagli incontri con l'azienda Proponente.

3.1.4.3 Redazione dell'Analisi dei Requisiti

A differenza degli altri artefatti_G, la redazione del documento di *Analisi dei Requisiti*_G è affidata esclusivamente all'*Analista*_G.

3.1.5 Verifica dei documenti

Ogni documento, a seguito della stesura o di un aggiornamento, deve essere sottoposto a verifica per accertare la correttezza e la coerenza delle informazioni riportate. Tale attività è svolta dal componente incaricato del ruolo di *Verificatore_G*.

3.1.5.1 Processo di verifica tramite Pull Request_G

Il gruppo adotta il meccanismo delle *Pull Request_G* (PR) tramite la piattaforma GitHub_G per garantire che nessuna modifica venga integrata nei branch_G principali senza un controllo preventivo. Il workflow_G operativo è così definito:

1. **Esecuzione modifiche:** Il redattore crea un branch_G locale. Le modifiche su documenti diversi devono essere gestite in branch_G separati.
2. **Apertura della Richiesta:** Completato il lavoro, il redattore esegue il **push** del branch e apre una PR, assegnando il *Verificatore_G* come reviewer e collegando la relativa *Issue_G*.
3. **Verifica:** Il verificatore_G esamina le modifiche:
 - **Esito Positivo:** Il verificatore approva la PR e procede al *merge_G*. Il branch_G viene eliminato.
 - **Esito Negativo:** Vengono inseriti commenti_G inline e lo stato viene impostato su *"Request changes"*.
4. **Risoluzione:** Il redattore applica le correzioni e richiede una nuova revisione fino all'approvazione.

3.1.6 Approvazione dei documenti

L'approvazione_G è l'atto formale che permette il rilascio di un documento come ufficiale nel branch *main*. Tale compito spetta al *Responsabile_G*.

3.1.6.1 Processo di approvazione tramite Pull Request_G

Al fine di evitare carichi eccessivi, i documenti vengono approvati progressivamente verso il raggiungimento di una *baseline_G*.

1. **Aggiornamento versione:** Il redattore crea un branch_G *chore/approvazione-<nome-documento>* modificando la versione nel frontespizio_G (incrementando l'indice Major X.0.0).
2. **Apertura Richiesta:** Si apre una PR assegnando il *Responsabile_G* come unico reviewer_G.
3. **Approvazione:** In caso di esito positivo, il responsabile approva il merge_G su *develop*. Successivamente, tutti i documenti approvati verranno uniti al branch *main*.

3.1.7 Nomenclatura dei documenti

3.1.7.1 Acronimi

Il gruppo adotta i seguenti acronimi standardizzati:

3.2 Gestione della configurazione

Questo processo di supporto si occupa dell'identificazione, attuazione e registrazione delle versioni di tutti gli elementi che, nel progetto, sono soggetti a modifiche che devono essere tracciate.

Acronimo	Significato
VI	Verbale Interno
VE	Verbale Esterno
NdP	Norme di Progetto _G
AdR	Analisi dei Requisiti _G
RTB	Requirements and Technology Baseline _G
PB	Product Baseline _G

Tabella 1: Tabella degli acronimi utilizzati nella documentazione.

3.2.0.1 Numeri di versione documenti

La convenzione scelta dal gruppo per il numero di versione dei documenti è lo standard *Semantic Versioning* MAJOR.MINOR.PATCH, X.Y.Z dove X rappresenta un'approvazione, Y rappresenta una modifica maggiore e Z rappresenta una modifica minore; in cui

- per **modifica minore** intendiamo l'aggiunta, la rimozione o la modifica di informazioni ritenute non significative (e.g. errori di battitura, di grammatica, sistemazione struttura sintattica dei periodi);
- per **modifica maggiore** intendiamo l'aggiunta, la rimozione o la modifica di informazioni ritenute significative (e.g. aggiunta di sezioni o paragrafi);
- per **approvazione** intendiamo far avanzare la cifra X solo e solamente quando il documento passa l'approvazione da parte del responsabile di sprint.

In caso di esito negativo della verifica, il documento non avanza di versione nel campo Minor o Major; le correzioni necessarie vengono tracciate tramite commit successivi all'interno del branch di lavoro. L'incremento di versione diventa effettivo e viene registrato nel frontespizio e nel registro delle modifiche solo una volta che il processo di verifica viene superato con successo.

Ogni documento nasce con versione 0.1.0 e viene aggiunto, tramite *merge_G*, nel main branch con versione X.Y.Z tale che $X > 0$ e $Y, Z = 0$.

3.2.1 Repository

Di seguito le *repository_G* create per la gestione della configurazione:

- **thebitless.github.io**: contiene tutta la documentazione di progetto e il sito web;
- **PoC**: contiene il sorgente del *Proof of Concept_G* del progetto;
- **MVP**: contiene il sorgente del *Minimum Viable Product_G* del progetto.

3.2.1.1 Strumenti

Di seguito gli strumenti adottati per la gestione della configurazione e del versionamento dei file di progetto:

- **Git_G**: sistema OpenSource di *VCS (Version Control System)* distribuito;
- **GitHub_G**: servizio connesso a Git che permette la condivisione di progetti software e l'hosting del sito web relativo al progetto.

3.3 Verifica

L'obiettivo della fase di verifica è accertare che gli output prodotti rispettino i criteri qualitativi e le specifiche delineate nel Piano di Qualifica, validando l'accuratezza e l'integrità di ogni consegna. Tali compiti sono affidati al componente del team che, nello sprint, svolge la funzione di verificatore. Quest'ultimo ha l'onere di documentare gli esiti delle analisi e, in presenza di difformità rispetto ai parametri prefissati, di sollecitare i necessari interventi correttivi. Se il carico di lavoro nello sprint lo richiede, il gruppo può aver assegnato più membri alla verifica.

Il gruppo applica ai seguenti prodotti il processo di verifica:

- **Documentazione:** ogni documento è sottoposto a verifica (si veda per dettagli ??) per verificare correttezza sintattica e semantica;
- **Codice sorgente:** una volta raggiunta la Requirements and Technology Baseline e resi disponibili i componenti software, si procederà a strutturare ed eseguire il piano di verifica. Questo iter comprenderà test basati su analisi statica e dinamica, le cui specifiche sono illustrate nelle sezioni che seguono.

3.3.1 Attività di verifica e implementazione del processo

Le attività di verifica agiscono sulle fasi dello sviluppo in modo che la verifica non sia solamente un controllo finale ma sia anzi un controllo costante della fase. In quest'ottica infatti ci si aspetta che il controllo finale verifichi solamente che il prodotto ottenuto al termine di una fase sia coerente con ciò che quella fase aveva avuto in input, ovvero non devono esser stati introdotti bug (per il codice) o refusi (per il testo).

Finora il gruppo si è occupato di documentazione e dunque la verifica verteva sulla correttezza sintattica, semantica e grammaticale di ogni documento. Per quanto concerne al codice, la verifica verrà trattata più approfonditamente dopo il raggiungimento della *RTB - Requirements and Technology Baseline_G* e le informazioni verranno riportate nel documento denominato **Piano di Qualifica_G**. La verifica è composta da due fasi:

- **Analisi Statica:** non richiede l'esecuzione del prodotto bensì verifica il prodotto tramite tecniche di analisi statica (es. controllo sintattico, semantico, grammaticale);
- **Analisi Dinamica:** richiede l'esecuzione del prodotto software in un ambiente controllato, ciò permette di effettuare dei test per valutare conformità ai requisiti funzionali e non funzionali.

3.3.1.1 Analisi Statica

Come già citato brevemente nella sezione ??, l'attività di Analisi Statica viene eseguita senza l'esecuzione del codice sorgente. Il suo obiettivo è individuare errori sintattici, violazioni degli standard di codifica e garantire la coerenza stilistica e grammaticale della documentazione. Questo permette di identificare i difetti precocemente nel ciclo di vita dello sviluppo. I metodi di analisi statica sono differenziati in:

- **Walkthrough:** consiste in una lettura informale del prodotto. Il verificatore esamina il documento o il codice alla ricerca di anomalie, producendo feedback per l'autore. È una tecnica orientata alla scoperta dei difetti e al miglioramento della qualità tramite il confronto diretto, utile anche per l'apprendimento tra i membri del team;

- **Inspection:** è una procedura di verifica formale e rigorosa, basata sull'utilizzo di checklist predefinite. Il verificatore esamina sistematicamente il prodotto per verificare la conformità a standard specifici, annotando ogni deviazione senza necessariamente suggerire una soluzione immediata.

Per la verifica dei documenti il gruppo ha preferito la procedura **Walkthrough** che permette al verificatore di controllare interamente un documento e di rilevarne incoerenza tra le sezioni o errori di tipo semantico/sintattico.

3.3.1.2 Analisi Dinamica

Come già citato brevemente nella sezione ??, l'attività di **Analisi Dinamica** richiede l'esecuzione del prodotto software in ambiente controllato. Esso utilizza i test, che devono essere:

- **Ripetibili:** per verificare se l'errore è stato corretto il test deve poter essere ripetuto in qualsiasi situazione;
- **Automatizzabili:** per evitare lo spreco di tempo, i test devono poter essere automatizzati tramite strumenti che simulano l'ambiente di testing;
- **Deterministici:** per poter garantire una valutazione oggettiva del risultato, ogni test deve avere un risultato atteso chiaramente definito;
- **Indipendenti:** ogni test deve poter essere eseguito in isolamento. L'esito di un test non deve essere influenzato dai risultati o dall'ordine di esecuzione degli altri test, garantendo che un fallimento sia localizzabile con precisione.

3.3.1.3 Test di Unità

I test di unità, in inglese *unit test_G*, sono quei test che valutano il corretto funzionamento della parte più piccola del software definita per il linguaggio di programmazione in cui quel software è stato codificato (e.g. funzioni, classi o metodi). Essi si dividono in:

- **Funzionali:** anche detti *black-box*, verificano solo l'output dell'unità senza indagare la logica interna;
- **Strutturali:** anche detti *white-box*, valutano anche la corretta esecuzione della logica interna senza concentrarsi solamente sull'output e si assicura che tutti gli statements rilevanti del codice vengano eseguiti almeno una volta.

3.3.1.4 Test di Integrazione

I test di integrazione, in inglese *integration test_G*, sono quei test che valutano l'interazione tra le varie componenti software e si usano primariamente per l'assemblaggio incrementale, ovvero in tutti quei momenti in cui si vuole integrare una nuova parte al sistema. Questo garantisce quello che in gergo si chiama *Continuous Integration_G*. I test di integrazione possono essere implementati con strategia **bottom-up** in cui si va ad integrare prima le componenti con minori dipendenze d'uso, oppure con strategia **top-down**, in cui si vanno ad integrare prima le componenti con maggiori dipendenze d'uso.

3.4 Validazione

Nella parte di validazione l'obiettivo è quello di controllare che il prodotto, sia esso software o documentale, corrisponda ai bisogni della proponente o, in fase avanzata, dell'utente. Questa fase viene svolta in presenza della proponente sfruttando i casi d'uso raccolti nell'*Analisi dei Requisiti_G* e conducendo test rappresentativi delle esigenze reali.

3.4.0.1 Attività previste

Le attività che prevediamo in questo processo sono:

- **Implementazione del processo;**
- **Esecuzione dei test;**
- **Raccolta dei risultati.**

3.4.0.2 Implementazione del processo

Come citati brevemente nella sezione ??, nel documento denominato *Analisi dei Requisiti_G* il gruppo ha delineato i requisiti che il prodotto deve rispettare. Andiamo quindi ad effettuare i seguenti test per controllare se i requisiti sono stati rispettati e se il prodotto risponde correttamente ai *Casi d'uso_G* definiti:

- **Test di sistema:** test funzionali sull'intero sistema;
- **Test di accettazione:** servono per accertare che i requisiti imposti dalla proponente siano rispettati.

3.4.0.3 Esecuzione dei test

L'implementazione dei test è guidata dai requisiti e dai casi d'uso cercando di immedesimarsi il più possibile nell'utente.

3.4.0.4 Raccolta dei risultati

I risultati dei test definiti qui sopra vengono raccolti nel documento denominato *Piano di Qualifica_G*, questo permette un'analisi dettagliata a confronto con i requisiti espressi.

4 Processi organizzativi

4.1 Processo di gestione

Il processo gestionale delinea le procedure e le mansioni comuni che ogni membro del gruppo deve adottare per assicurare il corretto avanzamento delle attività progettuali.

4.1.1 Pianificazione

4.1.1.1 Assegnazione delle responsabilità

Al fine di garantire una crescita professionale completa ed eterogenea, il team *The Bitless* ha adottato una politica di rotazione periodica. La riassegnazione dei ruoli avviene ogni **due settimane**, in corrispondenza dell'apertura di ogni nuovo *sprint_G*.

Di seguito l'elenco dei profili operativi previsti:

- **Responsabile** (Sezione ??)
- **Amministratore** (Sezione ??)
- **Progettista** (Sezione ??)
- **Analista** (Sezione ??)
- **Verificatore** (Sezione ??)
- **Programmatore** (Sezione ??)

4.1.1.2 Responsabile

Il *Project Manager_G* ha il compito di guidare il gruppo e fungere da interfaccia ufficiale verso i committenti. Deve possedere una visione d'insieme per valutare criticità e orientare le scelte strategiche.

- **Composizione:** È prevista la presenza di **un solo Responsabile** per ogni periodo.
- **Mansioni:**
 - Operare scelte decisionali a beneficio del team;
 - Validare i risultati prodotti dagli altri componenti;
 - Organizzare il calendario delle attività e allocare le risorse;
 - Gestire le relazioni e la comunicazione esterna.

4.1.1.3 Amministratore

Il *Sysadmin_G* è incaricato della configurazione e del mantenimento dell'ecosistema digitale (server, strumenti di lavoro, repository).

- **Composizione:** È prevista la presenza di al massimo due Amministratori per ogni periodo.
- **Mansioni:**
 - Individuare e implementare gli strumenti software necessari al rispetto delle *Norme di Progetto*;
 - Risolvere i malfunzionamenti tecnici segnalati tramite *ticket* (richieste di assistenza).

4.1.1.4 Progettista

Questa figura definisce l'architettura e le tecnologie del prodotto. Segue attivamente la fase di sviluppo, pur non essendo coinvolto direttamente nella manutenzione ordinaria.

- **Composizione:** Il ruolo può essere ricoperto da più membri simultaneamente.
- **Mansioni:**
 - Disegnare l'architettura di sistema per massimizzare prestazioni e scalabilità;
 - Monitorare l'implementazione del codice fornendo supporto tecnico ai programmatori.

4.1.1.5 Analista

L'Analista detiene la comprensione profonda del dominio applicativo e dei requisiti. Traduce le necessità del cliente in specifiche tecniche.

- **Composizione:** Per l'importanza strategica, questo ruolo coinvolge solitamente **più persone** nello stesso periodo.
- **Mansioni:**
 - Esaminare i bisogni dell'utente e il contesto del progetto;
 - Stabilire i traguardi operativi e la fattibilità delle richieste;
 - Redigere il documento di *Analisi dei Requisiti*.

4.1.1.6 Verificatore

Il Verificatore ha il compito di accertare che ogni prodotto sia conforme agli standard. Per garantire l'oggettività del processo, il membro incaricato della verifica di un'attività non può coincidere con colui che l'ha realizzata.

- **Composizione:** È un ruolo che può essere svolto da più membri contemporaneamente.
- **Mansioni:**
 - Controllare la coerenza degli *artefatti* (documenti e codice) rispetto alle *Norme di Progetto*;
 - Individuare e segnalare anomalie o refusi;
 - Elaborare test di validazione e di conformità ai requisiti.

4.1.1.7 Programmatore

È il profilo dedicato alla traduzione pratica dei requisiti in codice sorgente e alla successiva manutenzione.

- **Composizione:** Trattandosi della fase più onerosa in termini di tempo, è il ruolo che occupa solitamente il **maggior numero di componenti**.
- **Mansioni:**
 - Sviluppare le unità software seguendo le direttive dell'analista e l'architettura del progettista;
 - Produrre la documentazione tecnica (*Manuale*) del software.

4.1.2 Coordinamento

4.1.2.1 Riunioni interne periodiche

Il team *The Bitless* ha stabilito un incontro settimanale fisso (in presenza o telematico) per il confronto interno. Sebbene la partecipazione sia raccomandata, non è vincolante; tuttavia, in assenza della maggioranza, la sessione viene rinviata. L'incontro ordinario è fissato il martedì; ulteriori meeting straordinari possono essere concordati via chat.

4.1.2.2 Canali di comunicazione interna

Per lo scambio quotidiano di informazioni, documenti e comunicazioni rapide, il gruppo utilizza le seguenti piattaforme:

- *Discord_G*: Utilizzata come hub principale per la comunicazione asincrona e sincrona, sfruttando sia i canali testuali per la suddivisione tematica degli argomenti, sia i canali vocali per le sessioni di lavoro collaborativo;
- *WhatsApp_G*: Utilizzata per comunicazioni di carattere urgente o notifiche istantanee;

4.1.2.3 Comunicazione verso l'esterno

I rapporti con soggetti esterni avvengono tramite:

- **Email:** Utilizzando esclusivamente l'account ufficiale *thebitless.swe@gmail.com* per comunicazioni asincrone;
- **Google Meet:** Per meeting, presentazioni e videoconferenze in tempo reale.

4.1.3 Controllo e monitoraggio

4.1.3.1 Gestione dell'avanzamento

Questo processo vigila sulla coerenza tra l'esecuzione reale e la pianificazione, puntando all'ottimizzazione delle risorse. Il team monitora costantemente i progressi e l'impiego orario tramite strumenti dedicati. Le *Issue_G* (ovvero le singole unità di lavoro o problemi da risolvere) vengono organizzate all'interno di una *dashboard_G* di progetto che visualizza lo stato di avanzamento attraverso le seguenti colonne:

- **Backlog:** attività identificate ma non ancora pianificate per lo sprint corrente;
- **Ready:** attività pianificate e pronte per essere iniziate;
- **In Progress:** attività attualmente in corso di svolgimento;
- **In Review:** attività completate che necessitano di una verifica da parte del Verificatore;
- **Done:** attività verificate con successo e considerate concluse.

4.1.3.2 Gestione dei cicli di lavoro (Sprint)

Viene adottato un approccio *Agile* con *sprint_G* di quattordici giorni, articolati in quattro fasi fondamentali:

- **Sprint Planning:** Definizione dei target, scelta dei compiti dal backlog e stima dei tempi necessari;
- **Sviluppo:** Esecuzione operativa delle task (compiti) assegnate ai vari componenti;
- **Sprint Review:** Analisi dei risultati ottenuti alla fine delle due settimane e raccolta dei feedback;
- **Sprint Retrospective:** Esame critico dei processi interni per identificare e correggere eventuali inefficienze nel modo di lavorare del team.

4.1.3.3 Rilevazione delle ore

L'allocazione dei ruoli e il tempo effettivamente speso sono tracciati con precisione per garantire la trasparenza dei costi:

- **Piano di pianificazione:** Documento che riporta le ore previste e i ruoli assegnati a ogni membro del gruppo per ogni singolo sprint;
- **Registro consuntivo:** Documentazione delle ore realmente prestate da ogni membro durante ogni sprint, dato fondamentale per calcolare le metriche di progetto e l'efficienza del team.

4.1.3.4 Strumenti

Gli strumenti adottati per il controllo e il monitoraggio delle attività di progetto sono:

- **GitHub Projects:** piattaforma integrata con *GitHub_G* per la gestione delle Issue e la visualizzazione dell'avanzamento del progetto attraverso board personalizzabili;
- **Google Sheets:** strumento di fogli di calcolo condivisi per la pianificazione dei ruoli e il tracciamento delle ore lavorate da ciascun membro del gruppo.

4.1.4 Gestione dei rischi

4.1.4.1 Strategia di monitoraggio e contenimento

L'attività di **gestione dei rischi** è orientata all'individuazione precoce e alla neutralizzazione di quegli eventi incerti che potrebbero ostacolare il lineare avanzamento del progetto. Il gruppo *The Bitless* adotta una politica di **sorveglianza attiva**, mantenendo un presidio costante sulle criticità lungo l'intero ciclo di vita del prodotto.

La pianificazione operativa di tale processo è formalizzata all'interno del *Piano di Progetto*. Per ciascun rischio individuato, il documento analizza la probabilità di occorrenza e l'entità dell'impatto potenziale, definendo al contempo le procedure di prevenzione e le tattiche di mitigazione necessarie. Lo stato di salute dei rischi viene monitorato e aggiornato dinamicamente durante lo svolgimento degli *sprint_G*.

In fase di chiusura di ogni ciclo, durante la ***Sprint Retrospective***, il team esegue un'analisi retrospettiva degli eventi manifestatisi per validare l'efficacia delle soluzioni adottate. Questo confronto critico permette di introdurre tempestivamente **azioni correttive** e di perfezionare le strategie di difesa.